Read Heroes with Limit and Offset with FastAPI

When a client sends a request to get all the heroes, we have been returning them all.

But if we had **thousands** of heroes that could consume a lot of **computational resources**, network bandwidth, etc.

So, we probably want to limit it.

Let's use the same **offset** and **limit** we learned about in the previous tutorial chapters for the API.

> **Info**
>
> In many cases, this is also called **pagination**.

Add a Limit and Offset to the Query Parameters

Let's add `limit` and `offset` to the query parameters.

By default, we will return the first results from the database, so `offset` will have a default value of `0`.

And by default, we will return a maximum of `100` heroes, so `limit` will have a default value of `100`.

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero
```

We want to allow clients to set different `offset` and `limit` values.

But we don't want them to be able to set a `limit` of something like `9999`, that's over `9000`! 😱

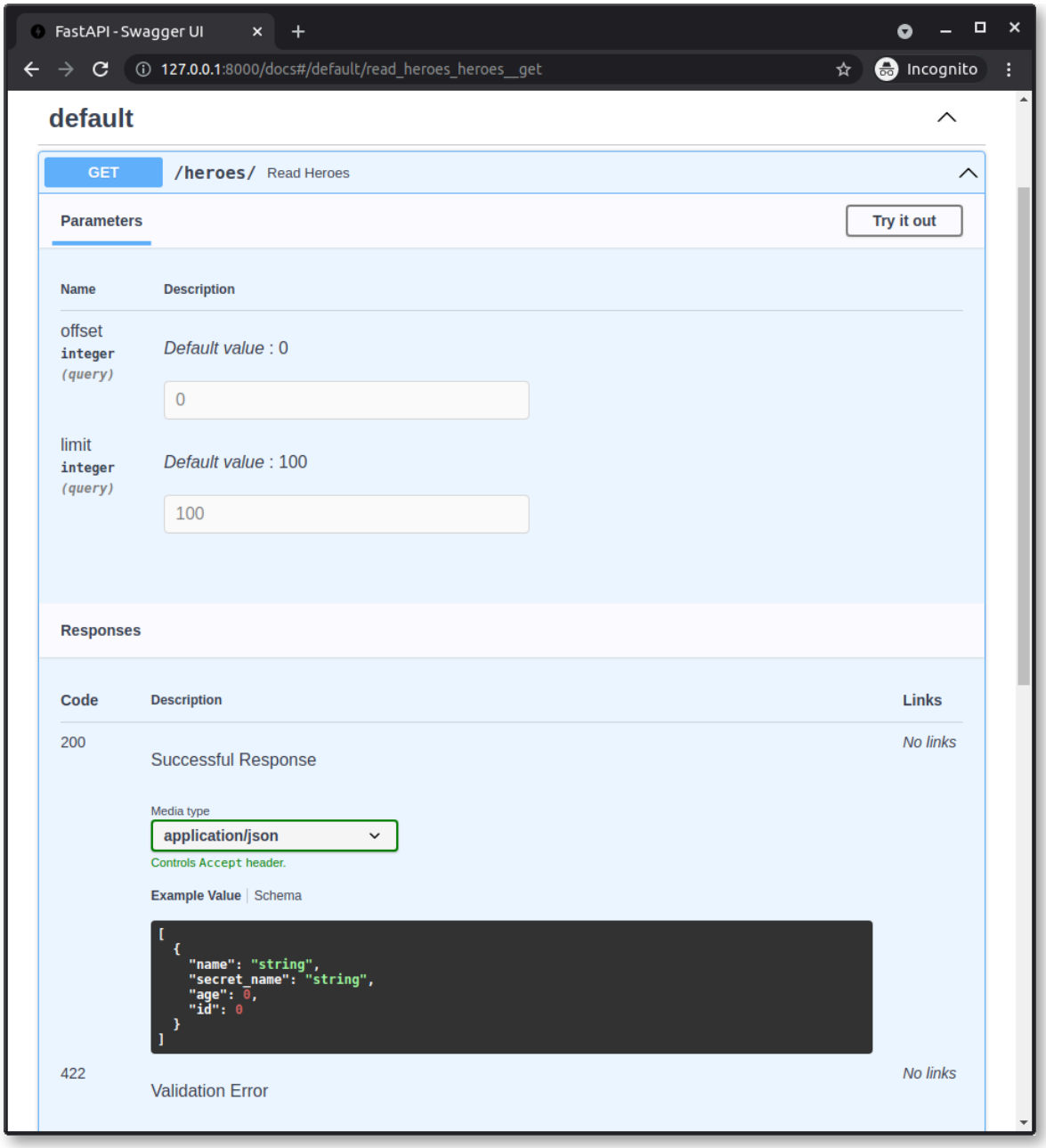
So, to prevent it, we add additional validation to the `limit` query parameter, declaring that it has to be **l**ess than or **e**qual to `100` with `le=100`.

This way, a client can decide to take fewer heroes if they want, but not more.

> **Info**
>
> If you need to refresh how query parameters and their validation work, check out the docs in FastAPI:
>
> - Query Parameters
> - Query Parameters and String Validations
> - Path Parameters and Numeric Validations

Check the Docs UI

Now we can see that the docs UI shows the new parameters to control **limit** and **offset** of our data.



Recap

You can use **FastAPI**'s automatic data validation to get the parameters for `limit` and `offset`, and then use them with the **session** to control ranges of data to be sent in responses.